

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

ARTIFICIAL INTELLIGENCE (AI)

Practical – VI

Roll No.: T009	Name: Siddhesh Kamble
Class:TYIT	Batch:1
Date of Assignment: 01/08/2026	Date/Time of Submission: 01/08/2026

Probability Reasoning (Bayesian Networks & Inference)

1. Using a simple example (Burglary–Alarm), create a Bayesian Network using pgmpy.

```

1. Using a simple example (Burglary–Alarm), create a Bayesian Network using pgmpy.

[17] ✓ 0.5s Python

# Bayesian Network using pgmpy
# Example: Burglary-Alarm Network

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

[2] ✓ 92s Python

c:\Users\mvluc\AppData\Local\Programs\Python\Python314\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter
from .autonotebook import tqdm as notebook_tqdm
c:\Users\mvluc\AppData\Local\Programs\Python\Python314\Lib\site-packages\pgmpy\estimators\_init_.py:4: FutureWarning: `pgmpy.estimators.Struct
from .StructureScore import (

# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm"),
    ("Alarm", "JohnCalls"),
    ("Alarm", "MaryCalls")
])

[3] ✓ 0.0s Python

# Step 2: Define Conditional Probability Tables

# P(Burglary)
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]}
)

# P(Earthquake)
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)

# P(Alarm | Burglary, Earthquake)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95] # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"]
    }
)

[4] ✓ 0.0s Python

```

```

    "Burglary": ["False", "True"],
    "Earthquake": ["False", "True"]
}
)
# P(JohnCalls | Alarm)
cpd_john = TabularCPD(
    variable="JohnCalls",
    variable_card=2,
    values=[
        [0.95, 0.10], # JohnCalls = False
        [0.05, 0.90] # JohnCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "JohnCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)
# P(MaryCalls | Alarm)
cpd_mary = TabularCPD(
    variable="MaryCalls",
    variable_card=2,
    values=[
        [0.99, 0.30], # MaryCalls = False
        [0.01, 0.70] # MaryCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "MaryCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)

# Step 3: Add CPDs to model
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm,
    cpd_john,
    cpd_mary
)

# Step 4: Check model correctness
print("Is Bayesian Network valid?", model.check_model())

# Step 5: Perform inference
inference = VariableElimination(model)
# Query: Probability of Burglary when John and Mary both call
result = inference.query(
    variables=["Burglary"],
    evidence={
        "JohnCalls": "True",
        "MaryCalls": "True"
    }
)

print("\nProbability of Burglary given JohnCalls=True and MaryCalls=True:")
print(result)
print("Siddhesh K(T009)")

```

Is Bayesian Network valid? True

```

Probability of Burglary given JohnCalls=True and MaryCalls=True:
+-----+-----+
| Burglary | phi(Burglary) |
+-----+-----+
| Burglary(False) | 0.7158 |
+-----+-----+
| Burglary(True) | 0.2842 |
+-----+-----+
Siddhesh K(T009)

```

2. Write a program to compute Posterior Probability using Bayesian

Networks(Burglar|Alarm = True)

2. Write a program to compute Posterior Probability using Bayesian Networks(Burglar|Alarm = True)

```
# Posterior Probability using Bayesian Network
# Query: P(Burglary | Alarm = True)

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
```

[9]

✓ 0.0s

Python

```
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm")
])
```

[10]

✓ 0.0s

Python

```
# Step 2: Define probability of Burglary
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]}
)
```

[11]

✓ 0.0s

Python

```
# Step 3: Define probability of Earthquake
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)
```

[12]

✓ 0.0s

Python

```
# Step 4: Define probability of Alarm
# P(Alarm | Burglary, Earthquake)

cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95] # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)
```

[13]

✓ 0.0s

Python

```
# Step 5: Add CPDs to model
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm
)
```

[14]

✓ 0.0s

Python

```
# Step 6: Check whether model is correct
print("Is Bayesian Network valid?", model.check_model())
```

[15]

✓ 0.0s

Python

```
... Is Bayesian Network valid? True
```

```

# Step 7: Perform inference
inference = VariableElimination(model)

result = inference.query(
    variables=["Burglary"],
    evidence={"Alarm": "True"}
)

print("\nPosterior Probability:")
print("P(Burglary | Alarm = True)")
print(result)
print("Siddhesh K(T009)")

```

[26] ✓ 0.0s Python

...

Posterior Probability:
P(Burglary | Alarm = True)

Burglary	phi(Burglary)
Burglary(False)	0.6264
Burglary(True)	0.3736

Siddhesh K(T009)

6A - Create a Bayesian Network for a real-world problem involving at least four variables using pgmpy.

6A — Titanic Passenger Survival: Create Bayesian Network

```

# Extra Task 6A
# Real-World Example: Hospital Patient Disease Diagnosis
# Bayesian Network using pgmpy

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

```

[28] ✓ 0.0s Python

```

# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Smoking", "LungDisease"),
    ("Age", "LungDisease"),
    ("LungDisease", "Cough"),
    ("LungDisease", "BreathingProblem"),
    ("LungDisease", "ChestPain")
])

```

[29] ✓ 0.0s Python

```

# Step 2: Define Conditional Probability Tables

# P(Smoking)
cpd_smoking = TabularCPD(
    variable="Smoking",
    variable_card=2,
    values=[[0.70], [0.30]],
    state_names={"Smoking": ["False", "True"]}
)

# P(Age)
cpd_age = TabularCPD(
    variable="Age",
    variable_card=2,
    values=[[0.60], [0.40]],
    state_names={"Age": ["Young", "Old"]}
)

# P(LungDisease | Smoking, Age)
cpd_lung = TabularCPD(
    variable="LungDisease",
    variable_card=2,
    values=[
        [0.95, 0.80, 0.70, 0.40], # LungDisease = False
        [0.05, 0.20, 0.30, 0.60] # LungDisease = True
    ],
    evidence=["Smoking", "Age"],
    evidence_card=[2, 2],
    state_names={
        "LungDisease": ["False", "True"],
        "Smoking": ["False", "True"],
    }
)

```

```

        "Smoking": ["False", "True"],
        "Age": ["Young", "Old"]
    }
)
# P(Cough | LungDisease)
cpd_cough = TabularCPD(
    variable="Cough",
    variable_card=2,
    values=[
        [0.80, 0.20], # Cough = False
        [0.20, 0.80]  # Cough = True
    ],
    evidence=["LungDisease"],
    evidence_card=[2],
    state_names={
        "Cough": ["False", "True"],
        "LungDisease": ["False", "True"]
    }
)
# P(BreathingProblem | LungDisease)
cpd_breathing = TabularCPD(
    variable="BreathingProblem",
    variable_card=2,
    values=[
        [0.90, 0.30], # BreathingProblem = False
        [0.10, 0.70]  # BreathingProblem = True
    ],
    evidence=["LungDisease"],
    evidence_card=[2],
    state_names={
        "BreathingProblem": ["False", "True"],
        "LungDisease": ["False", "True"]
    }
)
# P(ChestPain | LungDisease)
cpd_chestpain = TabularCPD(
    variable="ChestPain",
    variable_card=2,
    values=[
        [0.85, 0.40], # ChestPain = False
        [0.15, 0.60]  # ChestPain = True
    ],
    evidence=["LungDisease"],
    evidence_card=[2],
    state_names={
        "ChestPain": ["False", "True"],
        "LungDisease": ["False", "True"]
    }
)
)
[30] ✓ 0.0s Python

# Step 3: Add CPDs to model
model.add_cpds(
    cpd_smoking,
    cpd_age,
    cpd_lung,
    cpd_cough,
    cpd_breathing,
    cpd_chestpain
)
[31] ✓ 0.0s Python

# Step 4: Check model correctness
print("Is Bayesian Network valid?", model.check_model())
[32] ✓ 0.0s Python

... Is Bayesian Network valid? True

# Step 5: Display the Bayesian Network
print("\nBayesian Network Structure:")
print(model.edges())
[33] ✓ 0.0s Python

... Bayesian Network Structure:
[('Smoking', 'LungDisease'), ('LungDisease', 'Cough'), ('LungDisease', 'BreathingProblem'), ('LungDisease', 'ChestPain'), ('Age', 'LungDisease')]

```

```

# Step 6: Perform inference
inference = VariableElimination(model)
# Query: Probability of Lung Disease
# when patient has cough, breathing problem and chest pain
result = inference.query(
    variables=["LungDisease"],
    evidence={
        "Cough": "True",
        "BreathingProblem": "True",
        "ChestPain": "True"
    }
)
print("\nProbability of Lung Disease given:")
print("Cough = True")
print("Breathing Problem = True")
print("Chest Pain = True")
print(result)
print(result)

```

[34] ✓ 0.0s Python

...
Probability of Lung Disease given:
Cough = True
Breathing Problem = True
Chest Pain = True

LungDisease	phi(LungDisease)
LungDisease(False)	0.0339
LungDisease(True)	0.9661

6B - Perform inference to calculate a posterior probability for a given observation.

```

# Extra Task 6B
# Real-World Example: Loan Approval
# Query: P(LoanApproved | GoodCredit = True)

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

```

[37] ✓ 0.0s Python

```

# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("GoodCredit", "LoanApproved"),
    ("HighIncome", "LoanApproved")
])

```

[39] ✓ 0.0s Python

```

# Step 2: Define probability of Good Credit
cpd_credit = TabularCPD(
    variable="GoodCredit",
    variable_card=2,
    values=[[0.30], [0.70]],
    state_names={"GoodCredit": ["False", "True"]}
)

```

[40] ✓ 0.0s Python

```

# Step 3: Define probability of High Income
cpd_income = TabularCPD(
    variable="HighIncome",
    variable_card=2,
    values=[[0.60], [0.40]],
    state_names={"HighIncome": ["False", "True"]}
)

```

[41] ✓ 0.0s Python

```

# Step 4: Define probability of Loan Approval
# P(LoanApproved | GoodCredit, HighIncome)

cpd_loan = TabularCPD(
    variable="LoanApproved",
    variable_card=2,
    values=[
        [0.95, 0.70, 0.60, 0.10], # LoanApproved = False
        [0.05, 0.30, 0.40, 0.90] # LoanApproved = True
    ],
    evidence=["GoodCredit", "HighIncome"],

```

```

evidence_card=[2, 2],
state_names={
    "LoanApproved": ["False", "True"],
    "GoodCredit": ["False", "True"],
    "HighIncome": ["False", "True"]
}
)
[42] ✓ 0.0s Python

# Step 5: Add CPDs to model
model.add_cpds(
    cpd_credit,
    cpd_income,
    cpd_loan
)
[43] ✓ 0.0s Python

# Step 6: Check whether model is correct
print("Is Bayesian Network valid?", model.check_model())
[44] ✓ 0.0s Python

... Is Bayesian Network valid? True

# Step 7: Perform inference
inference = VariableElimination(model)
result = inference.query(
    variables=["LoanApproved"],
    evidence={"GoodCredit": "True"}
)
[45] ✓ 0.0s Python

# Step 8: Display posterior probability
print("\nPosterior Probability:")
print("P(LoanApproved | GoodCredit = True)")
print(result)
[48] ✓ 0.0s Python

...
Posterior Probability:
P(LoanApproved | GoodCredit = True)
+-----+-----+
| LoanApproved | phi(LoanApproved) |
+-----+-----+
| LoanApproved(False) | 0.4000 |
+-----+-----+
| LoanApproved(True) | 0.6000 |
+-----+-----+

```